

WorldForge 世界观工坊

版本：v0.1（需求规划版）

轻量 Wiki + 零切换写作 —— 本地优先（Local-First）的世界观创作管理软件

WorldForge 用来专门构筑作品的世界观与角色故事。它比 World Anvil 轻量、比 Obsidian 更结构化：用**卡片**管理角色 / 地点 / 事件 / 底层逻辑 / 参考资料，用**地图**表达方位与资源分布，用**时间轴**串起事件、角色年龄与文明演进，并在同一个界面里把正文写下去——写到哪，相关设定卡就跟到哪，鼠标悬停关键词即可预览。

所有数据保存在你自己的机器上：**不联网、不上传、不需要账号**。

快速开始

包管理器：**npm**（仓库只带 `package-lock.json`）。请勿混用 `pnpm` / `yarn`，否则 `node_modules` 的链接结构会被推平重装。

`package.json` 里的 `"allowScripts": { "esbuild": true }` 是 npm 11 的构建脚本审批白名单：
esbuild 需要执行 `postinstall` 准备平台原生二进制，Vite 的构建与开发服务器都依赖它。
因为已显式批准，`npm install` 时那条 `allow-scripts` 警告不会再次出现；换机器也不需要重新批准。

```
npm install          # 安装依赖
npm run dev          # 开发服务器（默认 http://localhost:5173）
npm run build        # 类型检查 + 生产构建 → dist/
npm run preview      # 预览生产构建
npm test             # 运行 239 项自测（9 个套件，Node 环境，无需浏览器）
npm run test:sample  # 由测试世界观手册重新生成 samples/ 里的可导入备份并校验
npm run icons        # 重新生成 PWA / Tauri 图标（零依赖脚本）
```

打包为桌面应用（需要 Rust 工具链）：

```
npm run tauri:dev    # 桌面端开发
npm run tauri:build  # Windows 安装包（NSIS + MSI）→ src-tauri/target/release/bundle/
```

安装到手机 / 平板：构建后用浏览器（或 Tauri 移动端）打开 `dist/`，
在「添加到主屏幕 / 安装应用」后即可作为 PWA 离线使用。

移动端打包：`npm run tauri android init` / `npm run tauri ios init`，鸿蒙通过 ArkWeb / 浏览器承载 PWA。

首次启动会写入一个**示例世界观「灰烬纪元」**（含 8 张卡片、5 条泳道、12 个时间轴条目、1 张地图、2 个资源区域、正文与大纲各一篇），可以直接在它上面改，也可以一键重建或清空（设置 → 数据 → 危险操作）。

第一次使用建议对着 `docs/使用说明.md` 走一遍：那里按「点哪个按钮 → 出现什么界面 → 选什么选项」写清了每个功能怎么用。

想把每个功能都喂上真实数据，用 docs/猫猫的冒险·世界观设定.txt：

那是一套完整的测试世界观（48 张卡片 / 7 个标签 / 16 条关联 / 2 张地图 / 5 条泳道 / 3 条平行世界分支），

可以照着文本手动录入，也可以直接导入 samples/猫猫的冒险.worldforge.json（同内容、机器可读）。

14 条需求与实现对照

#	需求	实现位置	说明
1	卡片式管理 + 信息互相关联、关系自由填写	src/features/cards/	9 种内置类型 + 插件可扩展；relations 表的 label 完全自由填写（如「师父」「导致」），支持双向反查与卡片详情页 [[标题]] 双链
2	可视化地图编辑器（方位、资源、政区、多张地图）	src/features/map/	底图 + 归一化标记点 + 可拖顶点多边形区域；人口/农业/矿产/军力/商贸资源条与「资源热度」着色；一张地图 = 一个时期，跨地图做时期对比
3	可视化时间轴（时长、年龄、状态、地理与科技变化）	src/features/timeline/	泳道分事件/角色生命线/地理变化/科技水平/底层设定；拖动游标即得「该时刻各角色年龄 + 状态 + 正在发生的事件」；数值型泳道画折线；条目可挂地图
4	世界观底层逻辑写作区	卡片类型 lore + 总览「底层逻辑」区	类别（魔法/修炼/神系/关键物质/世界规则/科技树...）、核心规则、代价限制、对世界的影响、演变趋势
5	插入图片	src/features/cards/CardGallery.tsx + plugins/gallery-lightbox.js	元数据入 SQLite、二进制入 IndexedDB；一张卡片多张图、可设封面；正文里可直接粘贴/拖入图片（asset：引用）；点图库任意一张图即全屏放大，可左右切换、键盘翻页、滚轮缩放
6	界面简约可隐藏、操作符合直觉、尽量可视化又保留纯文本	src/components/layout/	三栏（导航 / 侧栏 / 检查器）均可一键显隐 + 专注模式；Ctrl+K 命令面板；31 个键盘快捷键；可视化视图与 Markdown 文本视图随处可切
7	多端运行（Windows / 安卓 / 鸿蒙 / iOS）	src-tauri/ + public/manifest.webmanifest	同一份前端代码：Tauri 2 打包桌面与移动，PWA 覆盖其余平台；

#	需求	实现位置	说明
			响应式布局在手机上把侧栏变为抽屉、导航变为底部标签栏
8	平行世界设定	src/components/layout/WorldSwitcher.tsx + src/store/slices/dataSlice.ts	一个世界观下多条世界线； 主世界卡片在所有分支可见，分支可「派生」出独立副本； 可记录分歧点与分歧刻度；顶栏可切「全部分支」总览
9	参考资料区与关联	卡片类型 reference + 关联	资料类型/作者/链接/年份/关键摘录/ 可借鉴点；通过关联挂到任意角色、事件或设定上，卡片检查器反查「谁引用了我」
10	故事大纲区（可切树形）	src/features/outline/	与写作层同一套 Markdown 编辑器； 文本 / 树形 / 思维导图 三视图共享同一份数据， 可双向转换（按标题重建树、把树写回文本），节点可挂卡片、切状态、升降层级
11	tag 系统与筛选	src/features/cards/TagPicker.tsx	标签自由输入即建；多标签 AND 筛选；颜色可改；卡片列表/悬浮预览/时间轴处处可见
12	设定版本切换对比	src/features/versions/ + src/lib/snapshot-diff.ts	一键存快照； 随时与当前设定做 结构化 对比（新增/删除/修改的卡片、字段级变化、文稿行级 diff）；可整体还原、可导出单个快照
13	小巧不臃肿、构建快	见下方「体积」	无富文本编辑器、无图表库、无 UI 框架依赖；Markdown 解析器与 diff 均为自研；生产构建约 7 秒
14	插件区与插件插口	src/features/plugins/ + src/lib/plugin/	7 类插口（卡片类型、追加字段、面板、命令、卡片动作、主题、导出器）+ 事件订阅 + 只读查询 + 私有存储；附 4 个可读可改的内置示例插件，其中「图库灯箱」演示了不新增面板、直接给已有界面加交互的写法

技术栈与体积

React 18 + Vite 6 + TypeScript 5 + Tailwind CSS 3 + shadcn/ui(Radix) + sql.js(SQLite WASM)

- IndexedDB + Tauri 2 + PWA。

生产构建产物（`npm run build`）：

资源	原始	gzip
index.js（业务代码，含内置插件源码）	350 KB	101 KB
vendor-react	162 KB	54 KB
vendor-radix	109 KB	34 KB
vendor-sqljs	40 KB	14 KB
index.css	40 KB	8 KB
sql-wasm.wasm（按需加载）	644 KB	315 KB
合计（含按需加载的 wasm）	≈ 1.31 MB	≈ 527 KB

只算首屏必需的 JS + CSS 是 **≈ 0.68 MB / ≈ 211 KB（gzip）**；
SQLite 的 wasm 在初始化数据库时才拉取。表里的数字用 `gzip` 实测，
与打包器打印的 kB（十进制）略有差异。

刻意**没有**引入的重量级依赖：TipTap/ProseMirror（自研 Markdown 编辑器）、ECharts/Recharts
（地图、时间轴、思维导图、资源图全部手绘 SVG/DOM）、MUI/AntD、idb/dexie（自写 IndexedDB 封装）、
marked/markdown-it（自研解析器，顺便把 `[[双链]]` 与关键词自动关联做进解析流程）。

目录结构

src/	
├─ types/	领域类型：卡片字段字典、地图、时间轴、文稿、插件 API
└─ timeline.ts / card-types.ts / plugin-api.ts ...	
├─ lib/	
└─ db/	schema（分两部分）· sqlite 引擎 · init 初始化与迁移 · IndexedDB 封装
	表描述驱动的通用仓储 · 级联删除 · 保存状态
└─ markdown/	自研 Markdown 解析（块级 / 行内 / 双链 / 自动关联 / 统计）
└─ plugin/	插件注册表、宿主 API、事件总线、内置示例插件
└─ seed/	示例世界观数据（cards 设定层 / world 场景层）
└─ ...	diff、快照、快照差异、查询、大纲文本转换、素材、备份、ID、工具
├─ store/	zustand: ui / data / world / card / tag / map / timeline / doc /
	outline / version / plugin 十二个 slice
├─ components/	
└─ ui/	shadcn/ui 风格基础组件（button、dialog、tabs、select...）
└─ layout/	应用骨架、顶栏、图标导航、面板外壳、状态栏、命令面板
└─ common/	Markdown 编辑器 / 工具条 / 双链选择器 / 渲染视图 / 自动保存控件
├─ features/	board、cards、map、timeline、writer、outline、versions、plugins、settings
└─ hooks/	资源 URL、插件注册表订阅、快捷键、去抖
docs/	需求规格说明书、架构设计、数据模型、插件开发指南、使用说明、 测试世界观手册（猫猫的冒险 · 纯文本，可直接照抄录入）
samples/	由手册生成的测试数据备份（可用「设置 → 数据 → 导入设定」直接导入）
plugins/	以独立 .js 文件发布的插件源码（内置「图库灯箱」就在这里，可单独分享）
scripts/	图标生成 + 9 个自测套件（运行器、解析钩子、假 IndexedDB、selector / 表描述 / 插件静态检查器，均零依
src-tauri/	Tauri 2 壳（Cargo 配置、权限清单、入口）
public/	PWA manifest、Service Worker、图标

代码规范：每个文件不超过 200 行，注释说明「为什么这么做」而不只是「做了什么」。

数据与隐私

- 三层存储：**sql.js 内存 SQLite** → 去抖 800ms 导出二进制快照 → **IndexedDB**（kv 仓库）；
图片二进制单独存 IndexedDB 的 assets 仓库，SQLite 里只留元数据。
- 页面隐藏 / 关闭前强制落盘（visibilitychange + pagehide + beforeunload），状态栏实时显示保存状态。
- 可在「设置 → 数据」申请**持久化存储**授权，避免浏览器在空间紧张时清理数据。
- 导出：设定 JSON / 含图片的完整备份 / 原始 .sqlite 文件；导入会覆盖当前世界观内容（图片按 id 合并）。

质量验证

```
npm run typecheck # TypeScript 严格模式，0 错误
npm test          # 239 项自测（9 个套件，全部跑在 Node 里，不需要浏览器）
```

套件	项数	覆盖内容
scripts/selftest.mjs	12	Markdown 解析、HTML 转义防注入、外链协议白名单、[[双链]]、关键词自动关联（含不污染代码块）、行级 diff、超长文本退化路径、快照结构 diff
scripts/selftest-model.mjs	14	分支可见性、标签 AND 筛选、标题索引与长标题优先、关联双向反查、大纲文本 ⇌ 树、时间轴刻度与年龄推算、示例数据自治性
scripts/selftest-source.mjs	26	zustand selector 静态检查 （禁止在 selector 里 .filter() 等产生新引用的写法）+ 源码行数检查 （每文件 ≤ 200 行）+ Radix 面板检查 （TabsContent 上禁止 display 类，否则 hidden 失效、未激活面板仍占高度）；每项检查都以可证伪样本验证过有效性
scripts/selftest-specs.mjs	12	表描述 ↔ 领域类型一致性 （通用仓储用列名当属性名，字段名写错会静默失效）+ 插件 API 文档与源码一致 （指南里那行 worldforge:plugin-api 清单与 PluginAPI 强制比对）
scripts/plugin-selftest.mjs	11	插件源码检查 （幽灵 API、import、全局监听未移除、声明了却没人读的设置项、缺 activate /清理函数）+ 宿主 DOM 契约防漂移 （插件依赖的每个 data-wf-* 必须在宿主源码里真实存在）+ 内置清单接线 （?raw 引用的文件、manifest id 一致）+ api.settings 实时性 （行为测试：改完设置立刻读到新值）
scripts/sample-check.mjs	135	测试世界观数据自测 ：手册里的字段名与 card-types.ts 的 FieldDef 逐个对照、下拉取值必须在 options 内、id 引用与坐标刻度约束、手册声明的数量必须等于实际数量、 把生成的备份喂给真实 importBackup() 走一遍事务写库再读回核对 、手册指纹比对（改了手册忘记重新生成会直接报错）
scripts/db-selftest.mjs	13	建表 DDL（19 张表）、首次播种、落盘到 IndexedDB、卡片/标签/关联/地图/区域 CRUD、图库封面引用清理与装载自愈、插件记录字段完整性与往返
scripts/db-selftest-scene.mjs	10	时间轴条目（含瞬时事件判定、从卡片生成）、大纲树重建与回写、平行世界派生与清理
scripts/db-selftest-persist.mjs	6	版本快照 → 改动 → 还原、关闭应用后重新开库读回数据、级联删除不留孤儿数据

前 5 个套件是纯逻辑与静态检查，后 4 个套件使用**真实 sql.js**（SQLite WASM）+ 内存版 IndexedDB（scripts/fake-idb.mjs），通过模块解析钩子（scripts/alias-hook.mjs）把 @/ 别名与 Vite 的 ?url / ?raw 资源导入补齐，因此无需浏览器即可端到端验证数据层、内置插件源码的装载路径，以及导入备份的完整流程。

这些脚本在开发中抓到了真实缺陷。一例：版本还原时 saveMany 与外层事务嵌套，触发 SQLite cannot start a transaction within a transaction，导致「清空了但没写回」（修复方式：可重入事务，见 docs/数据模型.md §6.1）。另一例：**导入备份没有改写 world_id** —— 导出是按当前世界观做的，文件里带着来源世界的 id，导入到另一个世界观（例如新建一个空世界观再导入）时所有行仍属于来源世界，

界面上一条都看不到，而导入确认框已经把当前世界观清空了。

这个缺陷是写测试世界观数据时由 `scripts/sample-check.mjs` 的真实导入用例抓出来的。

代码规范自查：审计范围内共 **187 个源文件**，**没有任何文件超过 200 行**（`README` / 文档除外），该规则由 `scripts/selftest-source.mjs` 自动校验，不依赖人工统计。

⚠ 统计行数时不要用 PowerShell 的 `Get-Content / Measure-Object -Line`：
PS 5.1 读取无 BOM 的 UTF-8 文件会按 ANSI 解码，中文注释多的文件行数会**偏少**
（实测：91 行被算成 84 行、195 行被算成 187 行）。
正是这个偏差让一个 201 行的文件从「≤ 200 行」的人工审计中漏了过去，
所以现在改由上面的测试用 Node 按换行符精确统计。

v0.1 的边界（明确不做）

- 地图不做精细边界绘制与 GIS 投影，只做归一化标记与粗略多边形；
- 不做云同步、账号、多人协作；
- 插件无沙箱，属于**可信任模型**：请只载入自己写的或完全信任来源的插件；
- 版本快照不含图片二进制，也不支持分支级 diff 的自动合并；
- 时间轴使用数值刻度而非真实历法换算（用「刻度单位 + 年龄换算系数」近似表达）。

文档

文档	内容
<code>docs/使用说明.md</code>	用户手册 ：逐功能的分步操作（点哪个按钮 → 出现什么界面 → 选什么选项），含界面总览、快捷键、每个模块的完整流程、常见问题与已知边界
<code>docs/猫猫的冒险·世界观设定.txt</code>	测试世界观手册 （纯文本）：一整套「猫猫的冒险」设定，按 9 种卡片 / 标签 / 关联 / 地图 / 时间轴 / 文稿 / 大纲逐项列好，可照抄录入；由它生成的 <code>samples/猫猫的冒险.worldforge.json</code> 可直接导入
<code>docs/需求规格说明书-v0.1.md</code>	FR-01~FR-14 详述：用户故事、交互流程、数据落点、验收清单、范围边界、路线图、风险
<code>docs/架构设计.md</code>	分层架构、目录职责、九大关键机制、性能与体积预算、代码规范
<code>docs/数据模型.md</code>	19 张表字段级定义、ER 图、卡片字段字典、典型 SQL、事务可重入要点、迁移策略
<code>docs/插件开发指南.md</code>	PluginAPI 逐方法说明、事件与设置 schema、4 个示例（§3.4「图库灯箱」与实现逐字一致）、宿主 DOM 契约、发布与分享